Object-Oriented Programming (OOP, CCIT4023)

HKU SPACE Community College, 2022-2023

Assignment 4

(5%)

(Total Marks: 30)

- **Finish this work, based on concepts and techniques learnt in our course.** ***Submitted assessment work based on other non-taught concepts and techniques may be given zero mark or mark penalty.***
- **Students should finish reviewing the related course notes and materials, before doing this assignment.**
- **Individual work: Student MUST FINISH THIS WORK ALONE. Student cannot work with others.**
  ** Plagiarism / Collusion / Shared work with others are not allowed. Zero mark will be given, with possible disciplinary action.*
- Questions related to program codes are based on Java programming language, unless specified.
- Follow given instructions and guidelines.

Section A, A4A (10 marks): ONLINE (SOUL-Quiz Feature)

Multiple Choice (MC) and Matching Questions

- Identify and select the option of choice that "best" completes the statement, matches the item, or answers the question.
- **Number of Attempts Allowed: 2**
- **Grading method: Highest Grade**
- **Make sure you have *successfully* completed, "Finished" and submitted your work.**
  *** Attempts must be submitted before time expires, or they are NOT counted.**
- Questions related to program codes are based on Java programming language, unless specified.
- 10 MC questions and 10 Matching questions. Each question carries the same mark.

Section B, A4B (20 marks): Programming Tasks

**** Students may reference to the Practicing Labs to complete this task. Refer to the lab documents for details if necessary.***

General requirements (unless further specified):

- **Unless specified,** ***no data validation or specific exception handling required*** in the program (***program runs in normal situations***).
- **Unless specified, do not create your own package, do not add extras including methods and classes**.
- Unless specified, all related files (including `*.java`, `*.class`) should be working *in the same folder*.
- Each source code `*.java` file must ***start with comments***, including the information of individual student (***student name, student id, class number, year of study***) as example below, unless specified.

```
// A4B.java, For A4, OOP, 2022
/**
 * @author Student: <StudentName>, <SID>, <Class>, <Year>
 */
public class A4B{
```

Given Materials:

- This assignment document.
- Java files **`StudentInfoFileUtil.java`** and **`A4BkOrderSys.java`**, to be modified and completed by student. Also *modify the top comments for **your own Student Information***.
- Java file **`StudentInfo.java`** and **`BookOrder.java`**
  - ***DO NOT modify these given files.***
- For testing purposes: Text file `A4BStudents.txt`
  - ***DO NOT modify given Text file.***

A4B (20 marks)

Enhancing a Java Program to Model and Handle Information of *Book Order System* with File Handling

Write a Java program fulfilling the following section.

** This assessment work may not reflect a real and complete scenario in actual cases.*

StudentInfo
sID: int sName: String sPhone: int sPT: boolean
+ StudentInfo(id:int, name:String, phone:int, pt: boolean) + isValidPhone(): boolean + getSInfo(): String

BookOrder
bID: int bTitle: String bPrice: double bStudent: StudentInfo
+ BookOrder(id:int, title:String, price:double, student:StudentInfo) + dispBOInfo()

** Students may refer to Assignment 2 (A2) document for description details of these two given classes `StudentInfo` and `BookOrder`.*

Based on the earlier assignment and the given **`StudentInfo`** and **`BookOrder`** classes representing students and the related book orders, finish the following tasks.

1. **Class `ExtraBookOrder` *[8 marks]***

Create a public Java class **`ExtraBookOrder`**, in file `ExtraBookOrder.java`, to model an "Extra" book order (for data records) attached to an original existing `BookOrder` of the same student, based on the description below.

- This public class is a subclass of `BookOrder` class
- The class has its own specific ***field***:
  - A field, named `orgBookOrder` of type `BookOrder`, representing the original order this extra order attached to. This field can be accessed only within its own package and its subclasses.
- The class has ***two constructors***:
  - A private constructor, which accepts 4 input parameters in order as (`id, title, price, student`) of (id, book title, price and student of the order). This constructor simply calls the corresponding constructor of the superclass with these 4 input parameters accordingly.
  - One public constructor, which accepts 4 input parameters in order as (`id, title, price, orgord`) of (id, book title, price and the original book order attached to). This constructor calls another constructor of its own, with the first 3 parameters and the Student field of parameter `orgord` accordingly. It then sets its specific field with the input parameter `orgord`.
- The class has ***one overriding method***, named `dispBOInfo()` which accepts no parameter and no return value. This method displays the information of this book order on console.
  - The method body first displays two extra lines of information as the sample below, then calls the corresponding overridden method of its superclass as the format below:

```
.. An EXTRA Order, with Original Order ID: <bID>
... Total Amount: <Total sum of original and this extra>
Order ID: <bID>
Title: <bTitle>
Price(HKD): <bPrice>
Student: <sID>,<sName>,<sPhone>,<sPT??>
```

Example Program Output Format:

```
.. An EXTRA Order, with Original Order ID: 901
... Total Amount: 315.5
Order ID: 1231
Title: King Kong
Price(HKD): 90.5
Student: 20214321,CHOW Betty,88888888,PartTime
```

- The class has a "self-testing" method **`main()`** to run the class, and the method body does the following:
  - Display a message of self-testing (as sample output)
  - Create 3 `StudentInfo` objects, according to the information provided in Table 1(a)
  - Create an array of `BookOrder` of size 4, and assign array elements with objects according to the information provided in Table 1(b) and Table 1(c)
  - Display all `BookOrder` objects (*as sample output*)
  - Finally, display your student's information (name, id, class, year) before program terminates,

similar to the format as the sample output display on console below

Student ID	Name	Phone Number	Part-Time?
20210001	CHAN Tai Man	98765432	No
20214321	CHOW Betty	88888888	Yes
20215678	AU Candy	12000000	No

Table 1(a): StudentInfo

Order ID	Book Title	Price	Student
901	"Seven Little Plays"	225.0	CHOW Betty
902	"Treasure Island"	126.5	CHAN Tai Man

Table 1(b): BookOrder

Order ID	Book Title	Price	Original BookOrder (Order ID)
1231	"King Kong"	90.5	901
1232	"The Explorer"	173.5	902

Table 1(c): ExtraBookOrder

Sample Program Output, when self-testing: `java ExtraBookOrder`

```
*** ONLY FOR SELF-TESTING, ExtraBookOrder ***
Order ID: 901
Title: Seven Little Plays
Price(HKD): 225.0
Student: 20214321,CHOW Betty,88888888,PartTime

Order ID: 902
Title: Treasure Island
Price(HKD): 126.5
Student: 20210001,CHAN Tai Man,98765432

.. An EXTRA Order, with Original Order ID: 901
... Total Amount: 315.5
Order ID: 1231
Title: King Kong
Price(HKD): 90.5
Student: 20214321,CHOW Betty,88888888,PartTime

.. An EXTRA Order, with Original Order ID: 902
... Total Amount: 300.0
Order ID: 1232
Title: The Explorer
Price(HKD): 173.5
Student: 20210001,CHAN Tai Man,98765432


>>> END of main(). Done by <StudentName>, <SID>, <Class>, <Year> <<<
```

2. **Class `StudentInfoFileUtil` *[8 marks]***

Create a public Java class **`StudentInfoFileUtil`**, in file `StudentInfoFileUtil.java`, as a class supporting File handling of student records.

Text File Format for handling **`StudentInfo`**:

- Each record is stored with one line in CSV format *(ID, Name, Phone, PartTime or not)*, followed by a new line character

`"<sID>,<sName>,<sPhone><,sPT??>"`

  - E.g. for a full-time student **`"20210001,CHAN Tai Man,98765432"`**
  - E.g. for a part-time student **`"20210002,CHAN Siu Ming,98769876,PartTime"`**

Example Student Text File `A4BStudents.txt`, with 4 records.

```
20210001,CHAN Tai Man,98765432
20214321,CHOW Betty,88888888,PartTime
20215678,AU Candy,12000000
20211234,LAU Andy,55551111
20212021,TONG S.P.,82888828,PartTime
```

Write and complete 4 static methods below in this class (*DO NOT MODIFY the method header*):
* *NO specific exception handling required. If necessary, students may catch exceptions without doing any specific action or just printing a simple error message.*

- **`public static Student[] readStudFile(String rStudFile)`**: read student records from the text file `rStudFile`, and return an array of students
- **`public static void writePTStudFile(Student[] sArr, String wSFile)`**: write ONLY the part-time student records in the input array of students `sArr` into a text file `wSFile`
- **`public static void writeStudObjectFile(Student[] sArr, String wSObjF)`**: write all student records from the input array of students `sArr`, *directly as an array of `Student` objects* (**`Student[]`**) into an Object file `wSObjF`
- **`public static Student[] readStudObjectFile(String rSObjF)`**: read and return all student records, *directly as an array of `Student` objects* (**`Student[]`**) from an Object file `rSObjF`

3. **Class `A4BkOrderSys` *[4 marks]***

- **Create** a public Java class **`A4BkOrderSys`**, in file `A4BkOrderSys.java`, as the main class of this updated book order system (for data records).

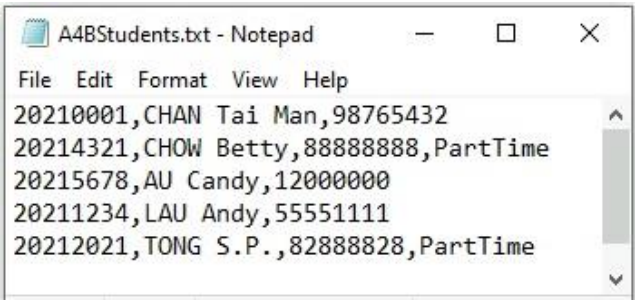
- Write and complete the following static method(s) in this program:
  - **`public static void main(String[] args)`** : the program entry point

- The method **`main()`** body does the following:
  - *Read data (and display on console) from the student text file* `"A4BStudents.txt"` as an array of students, with method `readStudFile` of above Java class `StudentInfoFileUtil`
  - *Write only the part-time students in the array of students above into the text file* `"A4BPTSts.txt"`, with method `writePTStudFile`
  - *Read data from the part-time student text* file `"A4BPTSts.txt"` as an array of students with method `readStudFile`, and *write this array directly to the part-time student Object file* `"A4BPTStObjects.obj"` with method `writeStudObjectFile`
  - *Read data (and display on console) from the part-time student Object file* `"A4BPTStObjects.obj"` as an array of students, with method `readStudObjectFile`
  - Finally, display your student's information (name, id, class, year) before program terminates, similar to the format as the sample output display on console below:

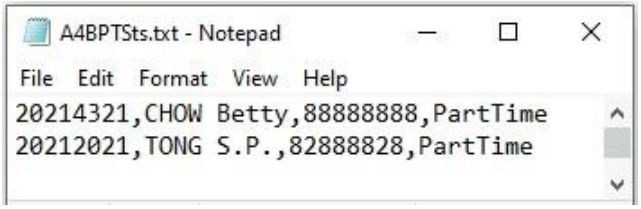
Sample Program Output: `java A4BkOrderSys`

```
--- A4 Book Order System, A4, OOP, 2022 ***
-1. READ (& Display) Student Text File: A4BStudents.txt
20210001,CHAN Tai Man,98765432
20214321,CHOW Betty,88888888,PartTime
20215678,AU Candy,12000000
20211234,LAU Andy,55551111
20212021,TONG S.P.,82888828,PartTime

--2. WRITE PT Student Text File: A4BPTSts.txt
 -- FINISHED: WRITE PT Student Text File: A4BPTSts.txt

---3. READ PT Student text File: A4BPTSts.txt; and WRITE PT Student
Object File: A4BPTStObjects.obj
 --- FINISHED: WRITE PT Student Object File: A4BPTStObjects.obj

----4. READ (& Display) PT Student Object File: A4BPTStObjects.obj
20214321,CHOW Betty,88888888,PartTime
20212021,TONG S.P.,82888828,PartTime

>>> END of main(). Done by <StudentName>, <SID>, <Class>, <Year> <<<
```

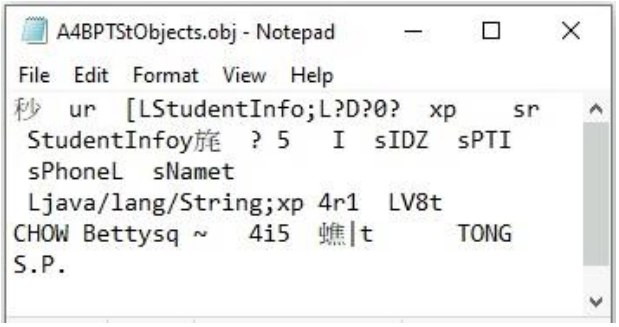
- Given Text File (`A4BStudents.txt`):



- Written Text File (`A4BPTSts.txt`):



- Written Object File (`A4BPTStObjects.obj`):



4. Evaluation and Submission

- Mark sure instructions are followed, including to follow the required *naming (e.g. parameters, methods and files)*.
- Compile, Run, Debug, Test and Evaluate your program based on the requirements.
- Submit ALL required files below to SOUL:
  - `ExtraBookOrder.java`, `StudentInfoFileUtil.java`, `A4BkOrderSys.java`, `StudentInfo.java`, `BookOrder.java`
  - `ExtraBookOrder.class`, `StudentInfoFileUtil.class`, `A4BkOrderSys.class`, `StudentInfo.class`, `BookOrder.class`
- **Do NOT compress/zip** or **rename** the files. Submission work not following instructions and requirements may not be marked or be penalized.
- ***IMPORTANT**** *No Late Submission. Make sure the submitted work is the most updated.*

**** Remarks:***

- *More testing may be performed during assessment based on the requirements, apart from the given testing cases.*

~ END ~